Python Engineering

WEEK 02 - Data Structures | CLASS 06 - Dictionaries & Sets | EX14

Cambio oficial de clase: dejamos class_05_lists_tuples y comenzamos **class_06_dictionaries_sets**. Seguimos dentro de week_02_data_structures.

EX14 - Configuracion y resultados mediante diccionarios

Un diccionario permite guardar informacion como pares **clave: valor**. En vez de recordar que state[0] es altura o que el tercer argumento era dt, podemos escribir nombres.

```
config = {
    'h0': 0.0,
    'v0': 50.0,
    't0': 0.0,
    'g': 9.81,
    'dt': 0.1,
}
```

1. Diccionario: clave -> valor

```
config['v0']      # 50.0
config['dt']      # 0.1
config['dt'] = 0.01
config['name'] = 'Euler run'
```

Los diccionarios son mutables. Las claves deben ser unicas: asignar de nuevo una clave actualiza su valor.

2. Metodos iniciales utiles

Expresion	Uso
d[key]	leer una clave existente
d.get(key)	leer sin fallar si no existe
d.keys()	vista de las claves
d.values()	vista de los valores
d.items()	pares (clave, valor)
key in d	comprobar si existe una clave
d.update({...})	actualizar varias claves

3. Set: conjunto de elementos unicos

```
required = {'h0', 'v0', 't0', 'g', 'dt'}
provided = set(config.keys())
```

Un **set** no mantiene duplicados y esta pensado para pertenencia y operaciones de conjuntos. No se accede mediante indices como en una lista.

4. Usar sets para validar configuraciones

```
missing = required - provided

if missing:
    print('Missing keys:', missing)
else:
    print('All required keys are present')
```

Aqui usamos una diferencia de conjuntos: elementos requeridos que no aparecen entre los proporcionados.

5. Operaciones principales de sets

Operacion	Python	Significado
Union	A | B	elementos de A o B
Interseccion	A & B	elementos comunes
Diferencia	A - B	elementos de A que no estan en B
Diferencia simetrica	A ^ B	elementos exclusivos de uno u otro
Pertenencia	x in A	x esta en el set?

6. Tu mision

Crea `ex14_simulation_config_dict_set.py`. No hace falta reescribir todo el simulador; queremos practicar estructuras.

```
1. Create a config dictionary with h0, v0, t0, g, dt.
2. Create a set with the required keys.
3. Convert config.keys() to a set.
4. Find missing keys using set difference.
5. If nothing is missing, read values from the dictionary.
6. Call your existing vertical-motion functions.
7. Store final outputs in a results dictionary.
8. Iterate over results.items() and print key/value pairs.
```

7. Results dictionary

```
results = {
    'max_altitude': max_h,
    'flight_time': t_f,
    'final_velocity': v_f,
    'iterations': i,
}
```

8. Test deliberado

Cuando funcione, elimina temporalmente la clave `'dt'` de config. Tu validacion con sets debe detectar exactamente esa clave como ausente antes de ejecutar la simulacion.

Learning Tracker

Concepto	EX14
dict y pares clave:valor	Nuevo
acceso / actualizacion por clave	Nuevo
keys(), values(), items()	Nuevo
set y elementos unicos	Nuevo
operaciones de conjuntos	Nuevo
validacion de campos requeridos	Nuevo
results dictionary	Nuevo